2.12 Neural Networks

Single Neuron

- linear transformation followed by a nonlin. transfo.
- a neuron receives input
  - For the first neuron $z_0 = x$
  - For subsequent neurons z_l, the input is output of previous neurons
- The neuron performs a computation:

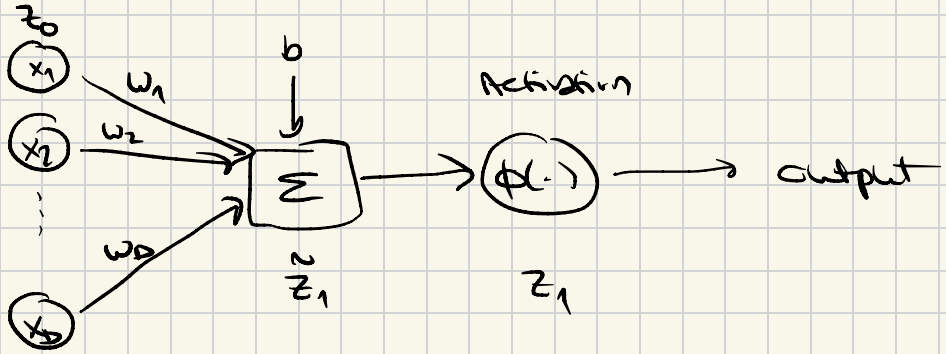
Step 1: "pre-activation"

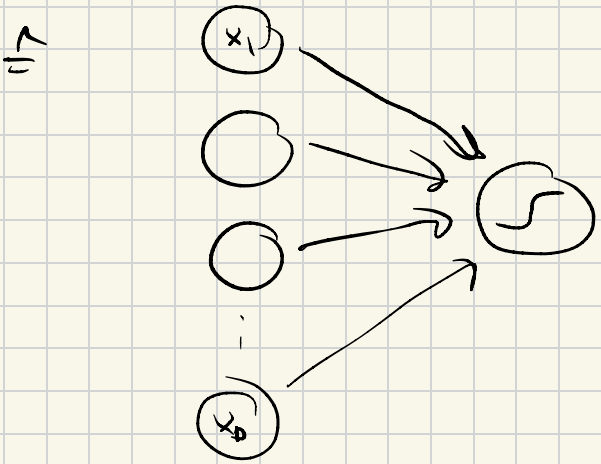
$$\tilde{z}_1 = W^T z_{(0)} + b$$

sometimes b is absorbed in W

Step 2: Apply nonlinearity "activation"

$$z_1 = \phi(\tilde{z}_1) \qquad \in \mathbb{R}$$





Popular Activation Function

1) Identity function

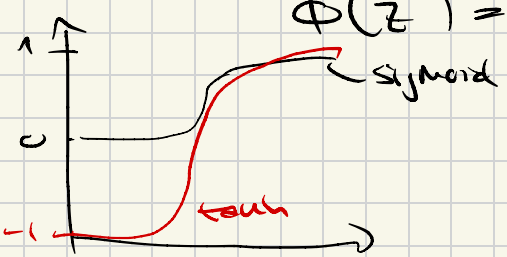
$$\phi(\tilde{z}) = \tilde{z}$$

2) Sigmoid

$$\phi(\tilde{z}) = \sigma(\tilde{z}) = \frac{1}{1+e^{-\tilde{z}}}$$

3) Hyperbolic tangent

$$\phi(\tilde{z}) = \tanh(\tilde{z})$$



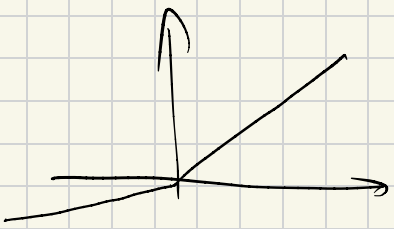
MODERN CHOICES

4) ReLU

$$\phi(\tilde{z}) = \text{ReLU}(\tilde{z}) = \max(0, \tilde{z})$$

5) Leaky ReLU

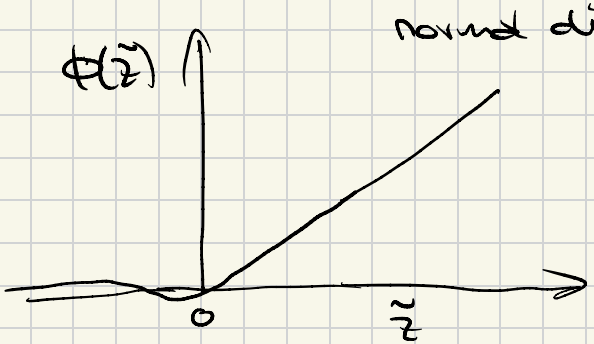
$$\phi(\tilde{z}) = \begin{cases} \tilde{z} & \text{if } \tilde{z} > 0 \\ \alpha \cdot \tilde{z} & \text{if } \tilde{z} \leq 0, \ \alpha > 0 \end{cases}$$



6) GELU (Gaussian Error Linear Unit)

$$\phi(\tilde{z}) = \text{GELU}(\tilde{z}) = \tilde{z} \cdot \varphi(\tilde{z})$$

φ = cumulative distr. fun of the normal distr.



Fully connected Networks

- Layer structure
  - "input" layer: Receives raw feature x

$$z_0 = x$$

  - "Hidden" layers: For layer $l = 1 \dots L-1$, the transformation is:

$$z_l = \phi_l\left(W_l z_{(l-1)} + b_l\right)$$

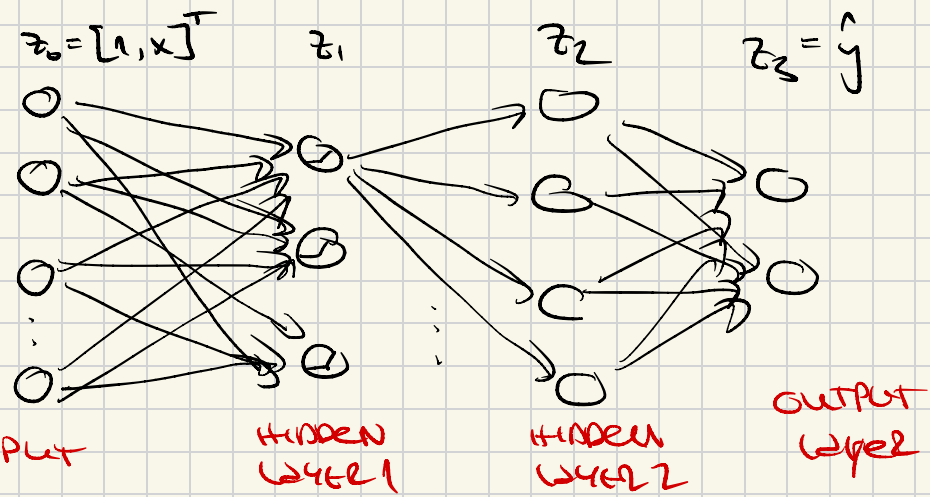
  - W_l is a weight matrix for layer $l \in \mathbb{R}^{d_l \times d_{l-1}}$
  - b_l is the bias vector $\in \mathbb{R}^{d_l}$
  - ϕ_l is the activation function, applied element-wise

  - "output" layer L: Typically, a linear transf. or softmax for classification produces the final output:

$$\hat{y} = z_L = W_L z_{L-1} + b_L$$

- A "deep neural network" (DNN) consists of multiple hidden layers of neurons

- A "fully connected network", a network whose each neuron in one layer is connected to every neuron in the next layer (Multilayer Perceptron (MLP))



$$f(x) = \phi_L\left(W_L \cdots \phi_2\left(W_2 \phi_1\left(W_1 x + b_1\right) + b_2\right) \cdots + b_L\right)$$

preactivation

activation

- For the L-th layer ϕ_L, we determine the activation function by application:

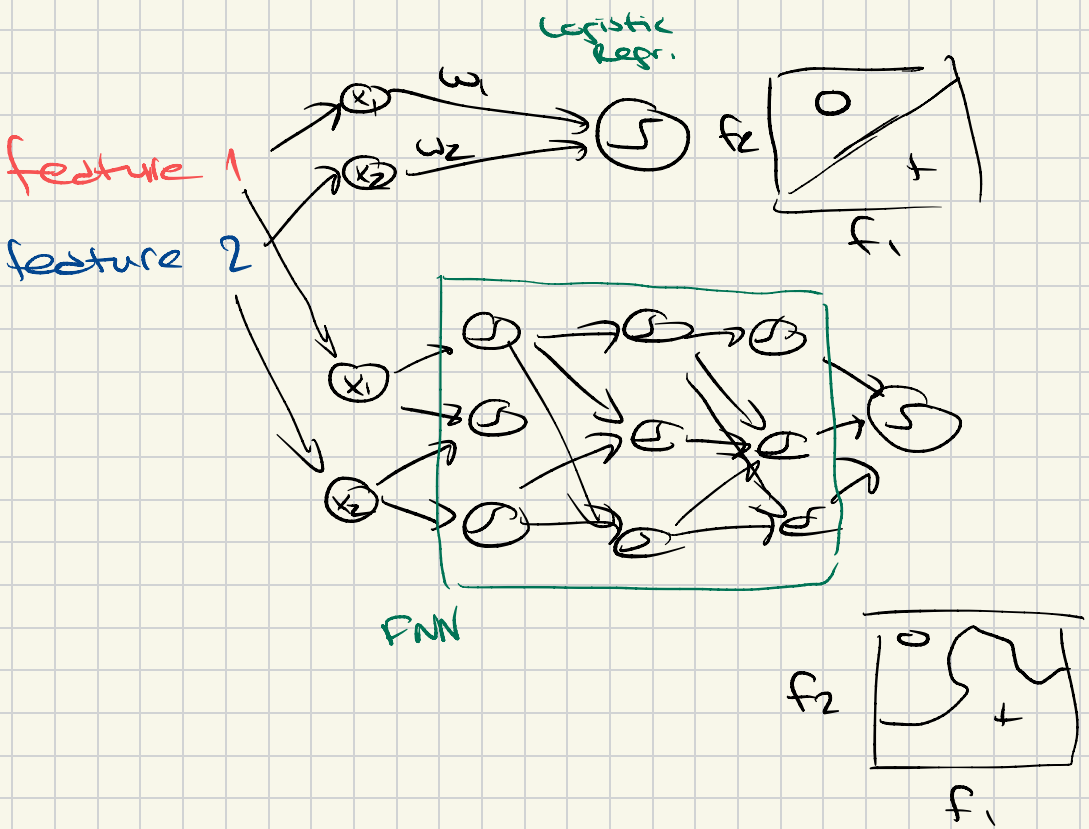
  1) Regression ($y \in \mathbb{R}$) : identity $\phi_L(\tilde{z}_L) = \tilde{z}_L$

  2) Classification $p(y=k|x)$: softmax

- Possible interpretation:

$$z_{L-1} = \psi([1, x]^T)$$

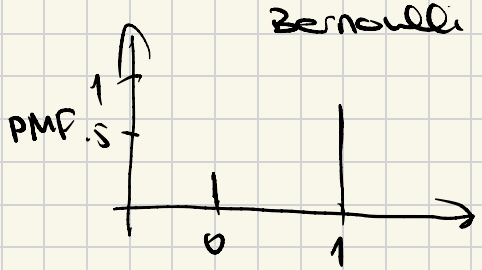
is a nonlin. transformation of the features



EXCURSION

$C = 2$

Bernoulli $\text{Ber}(\pi_i)$



$$p(y_i = 1) = \pi_i$$

$$p(y_i = 0) = 1 - \pi_i$$

PMF : $p(y_i) = \pi_i^{y_i} (1 - \pi_i)^{1 - y_i}$

$$E[y] = \pi_i$$

$$E[y \mid x] = \pi_i(x)$$

how to get π_i?

$$\pi_i = \frac{1}{1 + e^{-\vec{w} x_i}} \in (0, 1)$$

Not Bayesian

likelihood :

$$p_\theta(y_1, \dots y_N \mid x_1 \dots x_N) \overset{\text{iid}}{=} \prod_i^N p_\theta(y_i \mid x_i)$$

$$= \prod_i \pi_i^{y_i} (1 - \pi_i)^{1 - y_i}$$

FOR NN's, with $z^{(L)}$ being ow output of the last layer :

$$z_i^{(L)} = \text{FNN}(x_i)$$

$C > 2$

vector of probabilities

$\text{Cat}(\underline{\pi})$

$$y = \begin{pmatrix} 0 \\ 0 \\ 1 \\ c \\ 0 \end{pmatrix} \in \mathbb{R}^{C \times 1}$$

$$p(y_i = k \mid x_i) = \hat{\pi}_{ik} = \hat{y}_{ik}$$

$$\pi_{ik} = \frac{e^{\tilde{z}_{ik}^{(L)}}}{\sum_{k'} e^{\tilde{z}_{ik'}}} \in (0,1)$$

→ softmax

- likelihood function:

$$P_\theta(y_1, \dots y_N \mid x_1 \dots x_N) \overset{iid}{=} \prod_i^N p(y_i \mid \tilde{z}_i^{(L)})$$

$$= \prod_i \left(\prod_{k=1}^{C} \hat{\pi}_{ik}^{y_{ik}} \right)$$

- Take log, focus on one instance.

$$\ell(y_i, \hat{y}_i) = -\log \prod_{k=1}^{C} \pi_{ik}^{y_{ik}}$$

$$= -\sum_k y_{ik} \log \hat{\pi}_{ik}$$

cross-entropy loss

Historical remarks

1) efficient implementation of FNNs using graphic cards (GPUs)
2) big data
3) layer-wise pretraining (~2006, Hinton and colleagues)

Why are networks so good? 2 important theorems

1) Universal Function Approx. Theorem

Let $g: \mathbb{R} \to \mathbb{R}$ be a continuous, non-constant, and bounded function that satisfies

$$\lim_{z \to \infty} g(z) = 1 \qquad \lim_{z \to -\infty} g(z) = 0$$

Then for any continuous function on a compact set

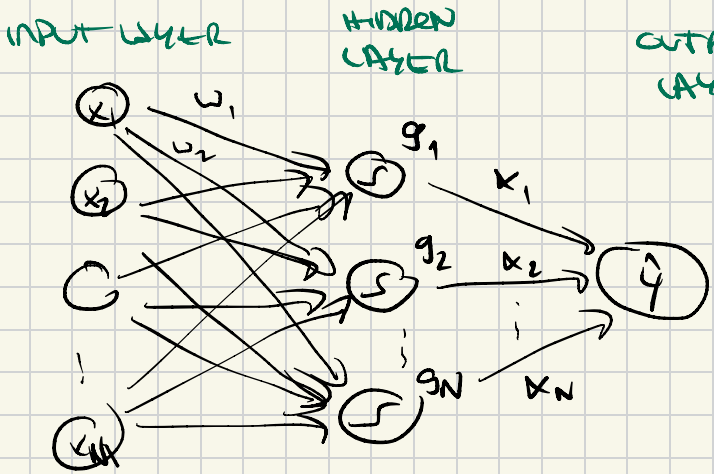
$$f \in \mathcal{C}(I_M) \quad \text{with } I_M = [0,1]^M$$

and for any $\varepsilon > 0$, there exist a finite sum

$$F_\theta(x) = \sum_{j=1}^{N} \alpha_j \, g(w_j^T x + w_{0j})$$

such that

$$|f(x) - F_\theta(x)| < \varepsilon, \quad \forall x \in I_M$$



$$\hat{y} = F_\theta(x)$$

⇒ Neural networks with at least two layers (one hidden layer) are universal function approximators.

- exact form: Cybenko (1989), Funahashi 1989
- Extension to other non-lin. functions, eg ReLU, hyperbolic tangent; Hornik 1991, Lu et al 2017
- Also applies to multivariate setting

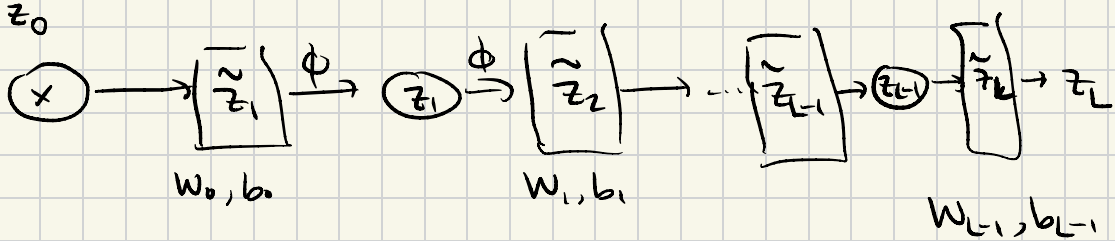
→ purely existential proof

2) Finding the optimal network is NP-hard.

Training with Backpropagation

- Idea: Train $W_0, W_1 \dots W_{L-1}$ by Gradient descent. Update rule:

$$W_\ell^{(t)} = W_\ell^{(t-1)} - \tau \frac{\partial \text{loss}}{\partial W_\ell^{(t-1)}}$$



$$\hat{y} = (\phi_L \circ \phi_{L-1} \circ \dots \phi_1)(x) = \phi_L(\phi_{L-1} \dots (\phi_1(x))$$

- Network structure:

$$z_0 := x$$

$$\tilde{z}_\ell := W_{\ell-1} z_{\ell-1} + b_{\ell-1}$$

$$z_\ell := \phi_\ell(W_{\ell-1} z_{\ell-1} + b_{\ell-1})$$

for $\ell = 1 \dots L-1$

- To obtain loss, distinguish between:

  1) regression: use squared loss

$$\mathcal{L}(\theta) = \frac{1}{2}\| y - z_L \|^2$$

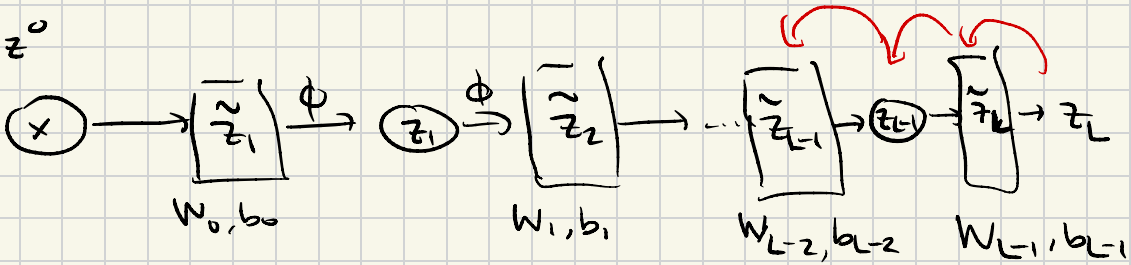
  2) classification: use cross-entropy loss

$$\mathcal{L}(\theta) = -\sum_k y_k \log(\hat{y}_k)$$

  for "one-hot" encoded y

- Goal: obtain $\theta = \{W_0, b_0, W_1, b_1 \cdots W_{L-1}, b_{L-1}\}$

- Apply chain rule:



$$\frac{\partial \mathcal{L}}{\partial \theta_{L-1}} = \frac{\partial \mathcal{L}}{\partial z_L} \frac{\partial z_L}{\partial \theta_{L-1}}$$

$$\frac{\partial \mathcal{L}}{\partial \theta_{L-2}} = \frac{\partial \mathcal{L}}{\partial z_L} \frac{\partial z_L}{\partial z_{L-1}} \frac{\partial z_{L-1}}{\partial \theta_{L-2}}$$

$$\frac{\partial \ell}{\partial \theta_l} = \frac{\partial \ell}{\partial z_L} \frac{\partial z_L}{\partial z_{L-1}} \cdots \frac{\partial z_{l+2}}{\partial z_{l+1}} \frac{\partial z_{l+1}}{\partial \theta_l}$$

$$= \frac{\partial \ell}{\partial z_L} \left(\prod_{j=i+1}^{L-1} \frac{\partial z_{j+1}}{\partial z_j} \right) \frac{\partial z_{l+1}}{\partial \theta_l}$$

$$\frac{\partial \ell}{\partial \theta_l} = \left(\frac{\partial z_{l+1}}{\partial \theta_l} \right)^T \left[\prod_{j=l+1}^{L-1} \left(\frac{\partial z_{j+1}}{\partial z_j} \right)^T \right] \frac{\partial \ell}{\partial z_L}$$

Example Regression

4 layer NN

$$z_1 = \phi_1 \underbrace{(W_0 x + b_0)}_{\tilde{z}_1}$$

$$z_2 = \phi_2 (W_1 z_1 + b_1)$$

$$z_3 = \phi_3 (W_2 z_2 + b_2)$$

output $\rightarrow$ $z_4 = \phi_4 \underbrace{(W_3 z_3 + b_3)}_{\tilde{z}_4}$

- loss: $\frac{1}{2} \| z_4 - y \|^2$ with $y \in \mathbb{R}^{d_4 \times 1}$

• step 1:

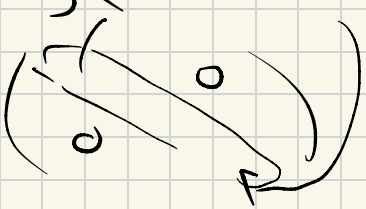
$$\frac{\partial R}{\partial W_3} = \underbrace{\left(\frac{\partial z_4}{\partial W_3}\right)^T}_{:=b} \underbrace{\frac{\partial R}{\partial z_4}}_{:=a}$$

a) $$\frac{\partial R}{\partial z_4} = z_4 - y \qquad \in \mathbb{R}^{d_4 \times 1}$$

b) $$\frac{\partial z_4}{\partial W_3} = \frac{\partial z_4}{\partial \tilde{z}_4} \frac{\partial \tilde{z}_4}{\partial W_3} = D\phi_4 \left(z^{(3)}\right)^T$$

$$z_4 = \phi(\tilde{z}_4) \qquad \tilde{z}_4 = W_3 z_3 + b_3$$

$$\frac{\partial z_4}{\partial \tilde{z}_4} = D\phi_4 = \operatorname{diag}\left(\phi_4'(\tilde{z}_4^{(1)}) \cdots \phi_4'(\tilde{z}_4^{(d_4)})\right)$$



$$\frac{\partial \tilde{z}_4}{\partial W_3} = \left(z_3\right)^T$$

(abuse of notation, really a tensor)

$$\frac{\partial \ell}{\partial W_3} = \left[D\phi_4 \, z_3^T \right]^T (z_4 - y)$$

$$= z_3 \, D\phi_4 (z_4 - y) \qquad \text{wrong dim}$$

$$\Rightarrow \frac{\partial \ell}{\partial W_3} = \overbrace{D\phi_4 (z_4 - y)}^{\delta^{(4)}} z_3^T$$

$\in \mathbb{R}^{d_4 \times 1}$ $\quad$ $\in \mathbb{R}^{1 \times d_3}$

- Step 2:

$$\frac{\partial \ell}{\partial W_2} = \underbrace{\left(\frac{\partial z_3}{\partial W_2} \right)^T}_{:=b} \underbrace{\left(\frac{\partial z_4}{\partial z_3} \right)^T}_{:=a} \underbrace{\frac{\partial \ell}{\partial z_4}}_{:=c}$$

$$z_3 = \phi_3(\tilde{z}_3) \qquad \tilde{z}_3 = W_2 z_2 + b_2$$

a)

$$\frac{\partial z_4}{\partial z_3} = \overbrace{\frac{\partial z_4}{\partial \tilde{z}_4}}^{D\phi_4} \underbrace{\frac{\partial \tilde{z}_4}{\partial z_3}}_{\frac{\partial}{\partial z_3} W_3 z_3 + b_3} = D\phi_4 W_3$$

- $\frac{\partial R}{\partial z_3} = \left(\frac{\partial z_4}{\partial z_3}\right)^T \frac{\partial R}{\partial z_4} = W_3^T D\phi_4 (z_4 - y)$

b)
- local derivative at layer 2:

$$\left(\frac{\partial z_3}{\partial W_2}\right)^T = \left(\frac{\partial z_3}{\partial \tilde{z}_3} \frac{\partial \tilde{z}_3}{\partial W_2}\right)^T$$

- $\frac{\partial z_3}{\partial \tilde{z}_3} = D\phi_3 = \text{diag}\left(\phi_3'(\tilde{z}_3) \dots \phi_3'(\tilde{z}_3^{d_3})\right)$

- Define the backpropagated error at layer 3:

$$\delta^{(3)} := D\phi_3 \frac{\partial R}{\partial z_3} = D\phi_3 W_3^T \cdot \delta^{(4)}$$

- $\frac{\partial \tilde{z}_3}{\partial W_2} = z_2^T$ (abuse of notation)

- Finally: $\frac{\partial R}{\partial W_2} = \delta^{(3)} z_2^T$

Summary

- In general, for a layer mapping:

$$z_{l+1} = \phi_{l+1}(\tilde{z}_{l+1}) \quad \tilde{z}_{l+1} = W_l z_l + b_l$$

  - $z_l \in \mathbb{R}^{d_l}$
  - $W_l \in \mathbb{R}^{d_{l+1} \times d_l}$
  - $\tilde{z}_{l+1} \in \mathbb{R}^{d_{l+1}}$

- Jacobian $\frac{\partial z_{l+1}}{\partial z_l} = D\phi_{l+1} \cdot W_l \in \mathbb{R}^{d_{l+1} \times d_l}$

- Backpropagated error

$$\delta^{(l+1)} = D\phi_{l+1} \frac{\partial R}{\partial z_{l+1}}$$

$$\frac{\partial R}{\partial z_l} = W_l^T \delta^{(l+1)}$$

$$\frac{\partial R}{\partial W_l} = \delta^{(l+1)} z_l^T$$

- In the classification case, the difference to regression is that we apply a softmax function

$$\hat{y} = z_L = \text{softmax}(\tilde{z}_L)$$

$$\hat{y}_i = \frac{e^{\tilde{z}_L^{(i)}}}{\sum_{j=1}^{d_L} e^{\tilde{z}_L^{(j)}}}$$

- For the cross-entropy loss

$$\mathcal{L} = -\sum_{i=1}^{d_L} y_i \log(\hat{y}_i)$$

one can show that:

$$\Rightarrow \quad \frac{\partial \mathcal{L}}{\partial z_L} = \hat{y} - y$$

- BP is the same for

1) regression problems w/ final linear layer and using the mean squared error (MSE) loss

2) classification problems w/ softmax layer and the cross-entropy loss

Algorithm: Backpropagation for an L-layer NN (softmax classification / linear regression)

- Input: Training instance (x, y); learning rate τ; parameters $\theta = \{W_i, b_i\}_{i=0}^{L-1}$
- Output: Updated parameters θ
- Forward pass:
  - set $z_0 \leftarrow x$
  - for $i = 1$ to $L-1$ do

    $\tilde{z}_i \leftarrow W_{i-1} z_{i-1} + b_{i-1}$

    $z_i \leftarrow \phi_i(\tilde{z}_i)$

    end for
  - $\tilde{z}_L \leftarrow W_{L-1} z_{L-1} + b_{L-1}$
  - $z_L \leftarrow \text{softmax}(\tilde{z}_L)$ $\quad z_L \leftarrow \tilde{z}_L$
  - Compute loss: $\ell \leftarrow \sum_i y_i \log(\tilde{z}_L^{(i)})$ $\quad \ell \leftarrow \frac{1}{2}\|z_L - y\|^2$
- Backward pass:

$\delta^{(L)} = z_L - y$

for $i = L-1$ downto 1, do

- compute weight gradients:

$$\frac{\partial R}{\partial W_i} \leftarrow \delta^{(i+1)} (z_i)^T$$

- compute bias gradients:

$$\frac{\partial R}{\partial b_i} \leftarrow \delta^{(i+1)}$$

- Propagate error:

$$\delta^{(i)} \leftarrow D_{\phi_i} W_i^T \delta^{(i+1)}$$

end for

$$\frac{\partial R}{\partial W_0} \leftarrow \delta^{(1)} x^T$$

$$\frac{\partial R}{\partial b_0} \leftarrow \delta^{(1)}$$

Parameter Update

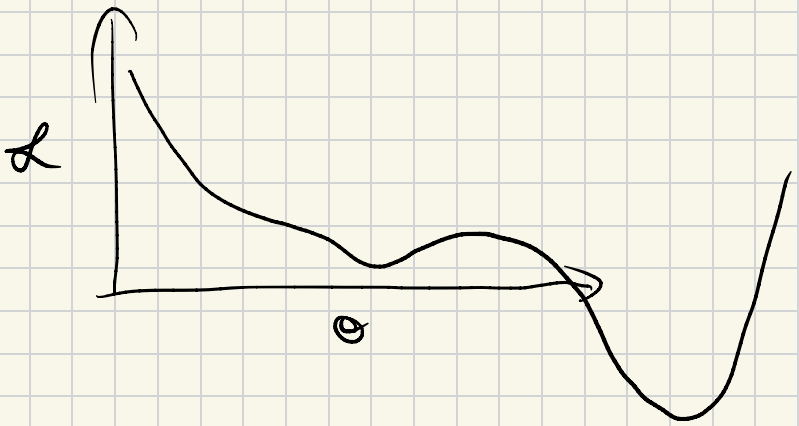
for $i = 0$ to $L-1$ do $:= \Delta W_i$

$$W_i \leftarrow W_i - \tau \frac{\partial R}{\partial W_i}$$

$$b_i \leftarrow b_i - \tau \frac{\partial R}{\partial b_i}$$

end for

2.13 NN training optimization



1) Momentum

- Consider a NN whose weights W_l you want to know
- ΔW_l → gradient update for this layer
- We maintain an expon. weighted moving average $\hat{=}$ momentum:

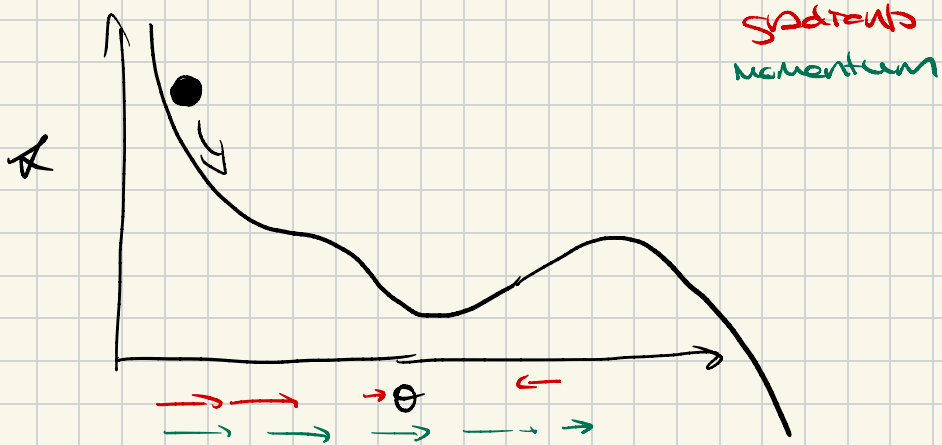
$$G_l^{(t)} = \mu G_l^{(t-1)} + (1-\mu)\Delta W_l$$

- $\mu \in [0,1]$, typical values $\mu = .9, .95 \ldots$
- $G_l^{(t-1)}$ is the accumulated momentum

from previous iterations

- Then update the weights using momentum

$$W_l^{(t)} = W_l^{(t-1)} - \tau G_l^{(t)}$$



- Problem: at the beginning of training, $G_l^{(t)}$ is biased towards 0, so we want to apply bias correction:

$$W_l^{(t)} = W_l^{(t-1)} - \frac{\tau}{1-\mu^t} G_l^{(t)}$$

Algorithm:

i) Initialize: For each layer l, $G_l^{(0)} = 0$

ii) For each iteration t:

a) compute ΔW_l

b) Update momentum:

$$G_l^{(t)} = \mu G_l^{(t-1)} + (1-\mu)\Delta W_l$$

c) Update parameters:

$$W_l^{(t)} = W_l^{(t-1)} - \frac{\eta}{1-\mu^t} G_l^{(t)}$$

2) Adam

- Use an adaptive learning rate
- Idea: Rescale the update to equalize the impact of all element of θ
- Compute two moving averages:

i) First moment (momentum) estimate:

$$\tilde{G}_l^{(t)} = \mu_1 \tilde{G}_l^{(t-1)} + (1-\mu_1)\Delta W_l$$

ii) Second moment Estimate:

$$\tilde{v}_l^{(t)} = \mu_2 \tilde{v}_l^{(t-1)} + (1-\mu_2)\left(\Delta W_l\right)^2$$

square is taken element-wise

- Possibly combine with bias correction:

$$G_l^{(t)} = \frac{\tilde{G}_l^{(t)}}{1-\mu_1^t} \qquad v_l^{(t)} = \frac{\tilde{v}_l^{(t)}}{1-\mu_2^t}$$

- Parameter update:

$$W_l^{(t)} = W_l^{(t-1)} - \frac{\tau}{\sqrt{v_l^{(t)}} + \varepsilon} G_l^{(t)}$$

- ε is a small constant

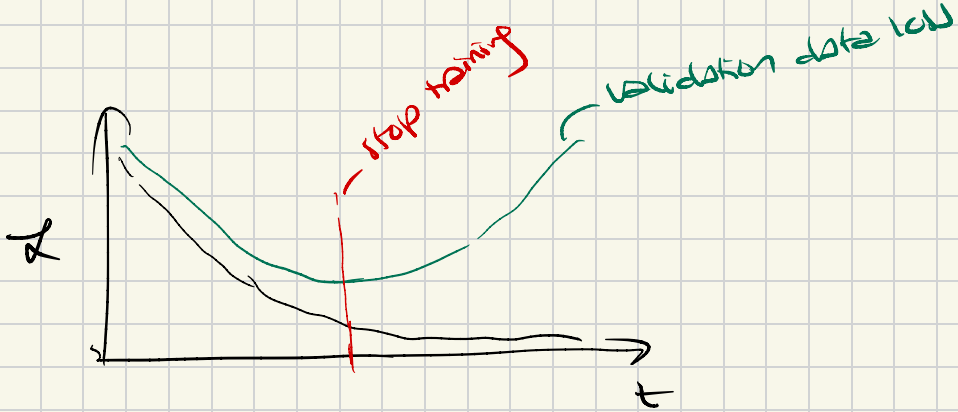
3) Learning rate schedule

- Let τ_t denote the learning rate at iteration t:

$$\tau_{t+1} = \gamma \tau_t \quad \text{if} \quad \mathcal{L}_{val} \geq \mathcal{L}_{val}^{(t-n)}$$

→ looking back for "n epochs"

4) Early stopping



- If for some t_0 and $\forall\, t \in \{t_0, t_{0+1}, \ldots, t_{0+p}\}$

$$\mathcal{L}_{val}^{(t)} > \min_{j<t} \mathcal{L}_{val}^{(j)}$$

then stop training